OpenCode.ai: The Open-Source AI Coding Agent for the Enterprise

The Executive Summary

OpenCode is an **open-source (MIT-licensed) AI coding agent** with 100K+ GitHub stars and 2.5M+ monthly active developers. Unlike GitHub Copilot or Claude Code, OpenCode is **model-agnostic** -- it connects to 75+ LLM providers including Claude Opus, GPT-5, Gemini 4, **and local open-source models via Ollama**. For enterprises, this means: use your best-in-class cloud model for complex tasks, use free local models for routine work, and **keep all data inside your network**. No vendor lock-in. No telemetry. No data leaving your org.

COST: HOW OPENCODE SLASHES PER-DEVELOPER SPEND

The Problem with Current Tooling

Tool	Cost/Developer/Month	What You Get
GitHub Copilot Business	$19	Autocomplete + limited agent mode, locked to GitHub's model selection
GitHub Copilot Enterprise	$39	Above + knowledge bases, Bing search
Cursor Pro	$20-200	IDE-locked, proprietary, limited model choice
Claude Code (Max)	$100-200+	Powerful agent, but Anthropic models only, cloud-dependent

A 50-developer team pays $12,000-120,000/year on AI tools -- plus the hidden cost of sending all your code to third-party servers.

The OpenCode Model: Pay Only for What You Use

Configuration	Cost/Developer/Month	When to Use
OpenCode + Local Models (Ollama)	**$0** (electricity only)	Routine code generation, boilerplate, refactoring, tests
OpenCode + Cloud API (Claude/GPT)	**$5-30** (pure API cost)	Complex architecture, multi-file changes, reasoning
OpenCode + GitHub Copilot sub	**$19** (existing sub)	Leverage your org's existing Copilot investment
OpenCode + Mixed (local + cloud)	**$3-15**	80% local for routine, 20% cloud for hard problems

Key insight: OpenCode itself is free. You pay only raw API costs -- no markup, no per-seat licensing for the tool itself. A mixed local+cloud strategy can reduce per-developer AI costs by **60-80%** compared to GitHub Copilot Enterprise.

50-Developer Team: Annual Cost Comparison

Approach	Annual Cost	Data Leaves Org?
GitHub Copilot Enterprise	**$23,400**	Yes -- all code sent to GitHub/Microsoft
Cursor Business	**$48,000+**	Yes -- sent to Cursor's servers
OpenCode + Local Models	**$0-3,000** (hardware amortized)	**No**
OpenCode + Mixed Strategy	**$3,600-9,000**	Only for cloud API calls (configurable)

MODEL FLEXIBILITY: USE ANY MODEL, ANYTIME

How OpenCode Integrates with Different Models

```
┌──────────────────────────────────────────────────────┐
│                    OpenCode Agent                    │
│         (Terminal / Desktop / IDE Extension)         │
├──────────────────────────────────────────────────────┤
│                   AI SDK + Models.dev                │
├──────────┬──────────┬──────────┬──────────────────┤
│  Claude  │  GPT-5   │  Gemini  │  GitHub Copilot  │
│  Opus    │  via     │  via     │  via existing    │
│  Sonnet  │  OpenAI  │  Vertex  │  subscription    │
├──────────┴──────────┴──────────┴──────────────────┤
│              LOCAL MODELS (Zero Cost)                │
│  Ollama │ llama.cpp │ LM Studio │ vLLM │ Docker MR  │
│  Qwen2.5-Coder │ DeepSeek R1 │ Llama 3.3 │ Phi-4   │
└──────────────────────────────────────────────────────┘
```

Key Integrations

Integration	How It Works	Why It Matters
Claude Opus/Sonnet	Direct Anthropic API or via AWS Bedrock	Best-in-class reasoning for complex tasks
GitHub Copilot	Uses existing Copilot subscription as a provider	Leverage your org's investment; no new procurement
Ollama (Local)	Points to `localhost` endpoint, zero network traffic	Routine tasks at zero cost, full air-gap capable
Docker Model Runner	Private inference inside Docker containers	Containerized, reproducible, enterprise-friendly
Azure OpenAI / Bedrock	Route through your org's approved cloud gateway	Compliance with existing cloud contracts

Recommended Multi-Tier Strategy

Task Type	Model	Cost	Latency
Autocomplete, boilerplate, tests	**Qwen2.5-Coder 32B (local)**	$0	~200ms
Code review, refactoring	**Llama 3.3 70B (local)** or Copilot	$0-19/mo	~500ms
Complex architecture, debugging	**Claude Opus / GPT-5 (API)**	~$0.05-0.30/task	~2-5s
Multi-file agentic changes	**Claude Opus (API)**	~$0.50-2.00/task	~10-30s

THE AGENTIC LAYER: WHY OPENCODE BEATS PURE GITHUB COPILOT

Copilot = Autocomplete. OpenCode = Agent.

Capability	GitHub Copilot	OpenCode
Inline autocomplete	Excellent	Not the focus (use Copilot for this)
Agentic multi-file edits	Limited (Agent Mode in IDE only)	Core strength -- plan, execute, iterate
Plan Mode	No	Yes -- propose changes before executing
Tool use (MCP)	Limited	Full MCP server support -- DB queries, API calls, custom tools
Custom skills/agents	No	Yes -- define reusable agent skills
Terminal-native workflow	No (IDE only)	Yes -- works in any terminal
Model choice	GitHub's selection only	75+ providers + local models
Undo/redo changes	IDE-level only	Built-in `/undo` `/redo` with full context
LSP integration	Via IDE	Native LSP server support
Open source	No	Yes (MIT) -- audit, fork, extend

The Best Strategy: Use Both Together

OpenCode and GitHub Copilot are **complementary, not competing**:

1. **GitHub Copilot** for real-time inline autocomplete as you type in your IDE
2. **OpenCode** for agentic tasks -- planning features, multi-file changes, debugging, code review
3. **OpenCode can use your Copilot subscription** as one of its model providers

This gives developers the best of both worlds: fast autocomplete + powerful agentic coding, with the flexibility to drop to local models when privacy or cost matters.

ENTERPRISE SETUP: ZERO DATA OUT

Architecture for Maximum Security

```
┌──────────────────────────────────────────────┐
│                YOUR NETWORK                  │
│                                              │
│  [Developer Machines]                        │
│       │                                      │
│       ▼                                      │
│  [OpenCode]  ──config──>  opencode.json      │
│       │                   • share: "disabled"  │
│       │                   • providers: internal  │
│       │                   • telemetry: off     │
│       │                                      │
│       ├──────> [Internal AI Gateway]         │
│       │        (Azure OpenAI / Bedrock / vLLM)  │
│       │                                      │
│       └──────> [Local Ollama / llama.cpp]    │
│                (Zero network traffic)        │
│                                              │
│  [SSO Provider] <── OpenCode auth integration  │
│                                              │
│          ╔═══════════════════╗               │
│          ║  NOTHING LEAVES    ║               │
│          ║  THIS BOUNDARY     ║               │
│          ╚═══════════════════╝               │
└──────────────────────────────────────────────┘
```

Enterprise Configuration Checklist

Setting	Configuration	Effect
Disable sharing	`"share": "disabled"` in config	No conversation data sent to opencode.ai
Restrict providers	Whitelist only internal gateway	All API calls route through your infra
SSO integration	Central config with SSO provider	Auth via your existing identity system
Disable external models	Block non-approved providers	Developers can only use approved models
Local-only mode	Ollama/llama.cpp on each machine	True air-gap -- zero outbound connections
Private npm registry	`.npmrc` with Artifactory/Nexus auth	No downloads from public npm
Central config deployment	Single `opencode.json` pushed org-wide	Consistent policy across all developers

Privacy and Telemetry: The Facts

Dimension	OpenCode	GitHub Copilot	Claude Code
Open source?	**Yes (MIT)** -- fully auditable	No	No
Built-in telemetry?	**None** -- no data collected or sent	Yes (usage data to Microsoft)	Yes (usage data to Anthropic)
Code sent to vendor?	**No** (tool itself sends nothing)	Yes (all context sent to GitHub)	Yes (all context sent to Anthropic)
Can be fully air-gapped?	**Yes** (with local models)	No	No
Share feature	Optional, can be disabled	N/A	N/A
Config auditable?	**Yes** -- JSON config, version-controllable	Partially	Partially

---

OPENCODE vs. ALTERNATIVES: QUICK COMPARISON

Feature	OpenCode	GitHub Copilot	Claude Code	Cursor
License	MIT (free)	$19-39/dev/mo	$100-200+/mo	$20-200/mo
Model lock-in	None (75+ providers)	GitHub models only	Anthropic only	Limited
Local model support	Full (Ollama, llama.cpp, vLLM)	No	No	Limited
Agentic coding	Full (plan, execute, tools)	Limited	Full	Moderate
MCP tool support	Yes	Limited	Yes	Limited
Enterprise SSO	Yes	Yes	Yes	Yes
Air-gapped operation	Yes	No	No	No
Data sovereignty	Complete	None	None	None
Terminal-native	Yes	No	Yes	No
Custom skills/agents	Yes	No	Limited	No

QUICK START: ENTERPRISE PILOT IN 15 MINUTES

```
# 1. Install OpenCode
npm install -g opencode

# 2. Create enterprise config (opencode.json in project root)
cat > opencode.json << 'EOF'
{
  "provider": {
    "name": "ollama",
    "baseURL": "http://localhost:11434/v1"
  },
  "share": "disabled"
}
EOF

# 3. Install Ollama + best coding model
brew install ollama
ollama pull qwen2.5-coder:32b

# 4. Launch OpenCode -- fully local, zero data out
opencode

# 5. (Optional) Connect to Claude Opus for complex tasks
# opencode /connect anthropic
```

USE CASE: THE "AI DEVELOPER" -- REGULATORY CODE ANALYSIS AT SCALE

The Problem

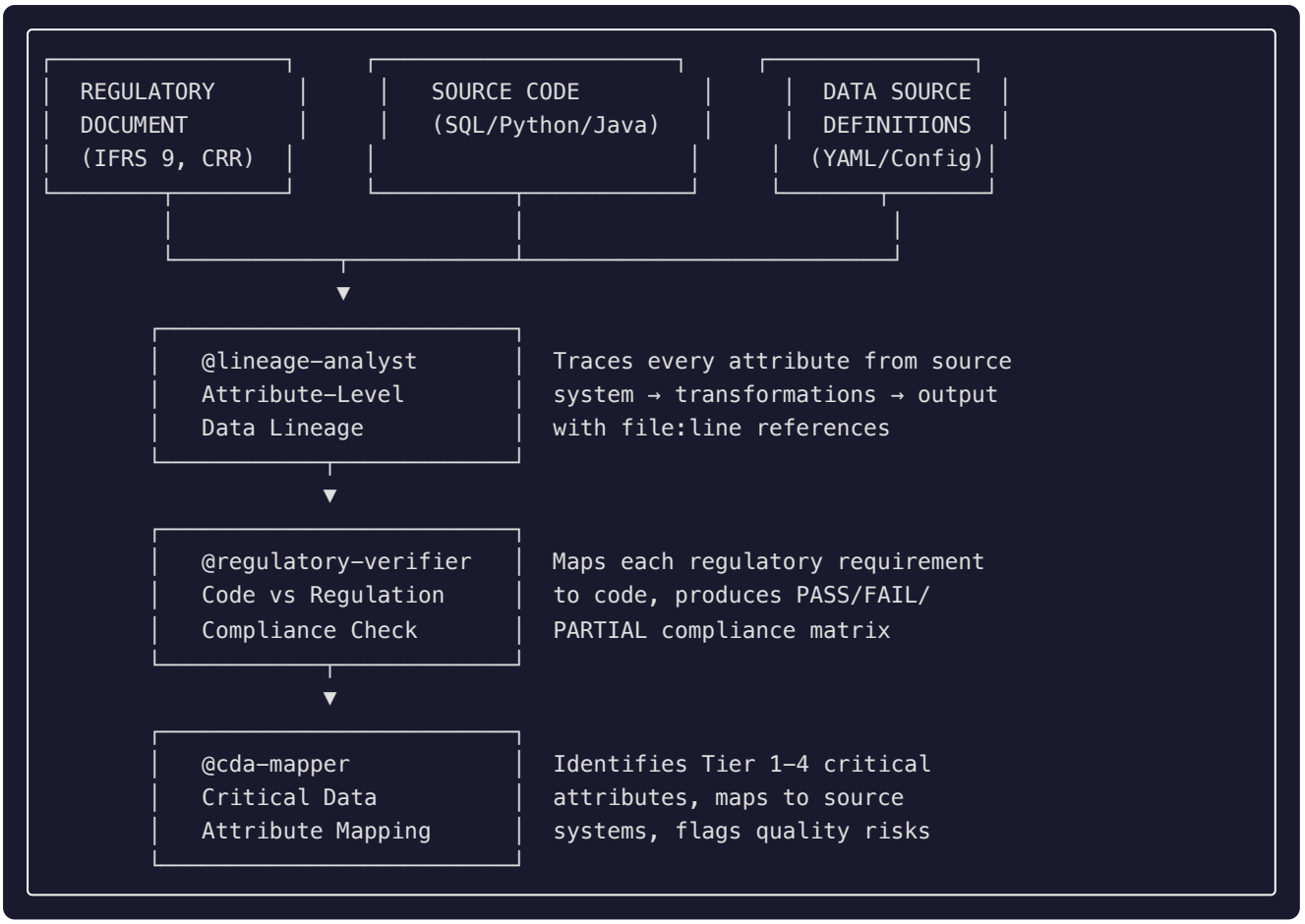
In financial institutions, **regulatory code analysis** is one of the most labor-intensive tasks:

- Building **attribute-level data lineage** for a single calculation takes 2-5 days manually
- **Verifying code** against regulatory documents (ECB, EBA, IFRS) requires deep dual expertise
- **Mapping critical data attributes** across source systems involves cross-team coordination
- Regulators (BCBS 239, ECB TRIM) increasingly demand automated, auditable lineage

A single regulatory calculation (e.g., IFRS 9 LGD) can involve 10-20 data attributes flowing through SQL, Python, and Java across multiple systems. Manual analysis is slow, error-prone, and expensive.

The Solution: AI Developer Agent Pipeline

OpenCode enables an **"AI Developer"** persona -- a suite of three specialized agents that work as a pipeline:



Worked Example: IFRS 9 LGD Calculation

Running the three agents against a Loss Given Default implementation:

Agent	Input	Output	Key Findings
@lineage-analyst	`lgd_calculation.py` + `.sql`	Lineage report: 8 attributes traced	5 complete, 2 hardcoded (haircuts, floors), 1 partial (ambiguous join)
@regulatory-verifier	IFRS 9 LGD requirements + code	Compliance matrix: 8 requirements	3 PASS, 1 FAIL (downturn model), 3 PARTIAL, 1 NOT FOUND (segmentation)
@cda-mapper	Regulation + code + data sources	CDA mapping: 13 attributes	4 missing inputs (workout costs, time-to-resolution), guarantee_amount fetched but unused

Critical finding detected: The regulation requires downturn LGD linked to macroeconomic conditions. The code uses a fixed 8% add-on -- a material compliance gap that manual review missed.

ROI: AI Developer vs Manual Analysis

Metric	Manual (Senior Analyst)	AI Developer (OpenCode)	Improvement
Lineage per calculation	2-5 days	10-30 minutes	**40-100x faster**
Regulatory verification	3-7 days	15-45 minutes	**30-70x faster**
CDA mapping	2-4 days	10-30 minutes	**40-80x faster**
Full pipeline (lineage + verification + CDA)	1-3 weeks	**1-2 hours**	**40-60x faster**
Cost per calculation analysis	$3,000-10,000	$0.50-5.00 (API) or $0 (local)	**99%+ reduction**
Consistency	Analyst-dependent	Deterministic templates	Auditable, repeatable

Enterprise Agent Templates (Included)

A complete project scaffold is provided in `ai-developer-agents/` :

Component	What It Contains
3 Agent definitions	`lineage-analyst.md` , `regulatory-verifier.md` , `cda-mapper.md` -- ready to use
3 Skills	`/trace-lineage` , `/verify-regulation` , `/map-cda` -- for quick single-attribute tasks
3 Output templates	Structured Markdown templates for lineage, verification, and CDA reports
Worked example	IFRS 9 LGD: regulation excerpt, Python + SQL code, data source definitions
Expected outputs	Complete sample reports showing what the agents produce
Enterprise config	`opencode.json` pre-configured for air-gapped operation

Supported Regulatory Frameworks

Regulation	Issuer	Typical Agent Use Cases
IFRS 9	IASB	ECL, PD, LGD, EAD lineage and verification
CRR/CRD (Basel III/IV)	ECB/EBA	RWA, capital ratio calculation validation
BCBS 239	BCBS	Data lineage compliance, aggregation quality
FRTB	BCBS	Sensitivity calculation, bucket assignment mapping
AnaCredit	ECB	Credit data attribute mapping to source systems
FINREP/COREP	EBA	Reporting attribute lineage and completeness

THE BOTTOM LINE

What	Why It Matters
60-80% cost reduction	Free tool + local models for routine work, cloud APIs only when needed
Zero vendor lock-in	Switch between Claude, GPT, Gemini, or local models in one config change
Complete data sovereignty	Open-source, no telemetry, air-gapped capable, enterprise config controls
Complementary to Copilot	Use Copilot for autocomplete + OpenCode for agentic tasks -- both together
Agentic > Autocomplete	Plan mode, MCP tools, multi-file edits, custom skills -- a coding agent, not a suggestion engine
Enterprise-ready	SSO, central config, provider whitelisting, private registries, share disabled

Recommendation: Run a 2-week pilot with 5-10 developers. Install OpenCode + Ollama (Qwen2.5-Coder 32B) alongside existing GitHub Copilot. Measure: tasks completed, time saved, cost per developer, and developer satisfaction. No procurement needed -- it's free and open source.

Document generated March 2026. Sources: OpenCode.ai, OpenCode GitHub, OpenCode Enterprise Docs, OpenCode Providers, Docker + OpenCode Integration, GitHub Copilot vs OpenCode, InfoQ Coverage, YUV.AI OpenCode CLI Guide.